TRACK C · CAPSTONE WALKTHROUGH

Real-Time Analytics Platform

Ingest events at volume. Answer funnel, retention and revenue questions in milliseconds.

PostgreSQL 16 | Redis Streams | Row-Level Security | Prometheus + Grafana

[Abdikafi mohamud abdi] · Database capstone

THE BRIEF

Three product questions, answered fast

Funnels

Where do users drop off between signup and purchase?

Retention

Do users come back after 7 and after 30 days?

Revenue

What does each signup cohort pay, by plan?

10M+ events in the benchmark

Dashboard queries under 1 second

Strict isolation between tenants

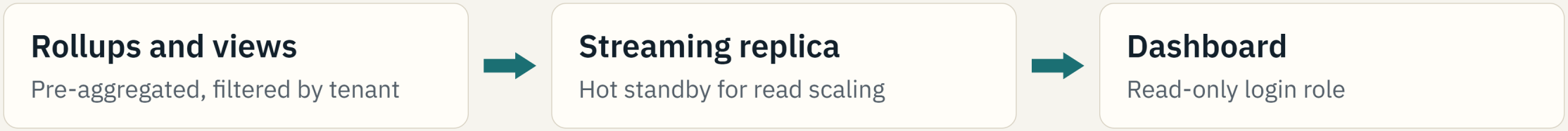
DESIGN

From ingestion to dashboard

WRITE PATH



READ PATH



Monitoring (configured): postgres and redis exporters feed Prometheus and Grafana.

DESIGN

A data model built for isolation and scale

Shared tables, tenant first

One schema for every tenant. The tenant id leads each key and index, and Row-Level Security filters every query.

Append-only events

Partitioned by month. Variable attributes live in JSONB with a GIN index, so no schema change per event type.

No foreign keys on events

Skips per-row checks on the hottest table. The consumer validates tenant and event type instead.

Rollups outlive raw data

Daily and per-user aggregates keep history after old raw partitions are dropped.

10 core tables plus 5 derived objects. The full ER diagram is in the repository docs.

DESIGN

Monthly partitions: fast queries, instant retention



Grey: past the retention window. Dark: current month, receives writes. Outline: created ahead. Dashed: safety net for out-of-range events.

BRIN index

Tiny index on time-ordered data. A time-range scan went from 175 ms to 48 ms.

Keep 90 days

Old months are dropped whole: instant, no huge DELETE. Archived with pg_dump first.

Safe by design

A month is never dropped before the rollups have consumed it.

EVIDENCE · INGESTION

Redis Streams absorbs the burst

16,833/s
200,000 events queued in Redis

~6,900/s
Written into PostgreSQL by one consumer

164,000
Peak backlog, drained to zero

0 lost
200,000 in, 200,000 stored, none dead-lettered

At-least-once delivery: acknowledge only after the commit. Invalid events go to a dead-letter stream.

EVIDENCE · QUERIES

Before and after, on 10.3M events

Query	Technique	Before	After	Faster
Q1 Funnel conversion	Covering index	1,108 ms	62 ms	18×
Q2 Rolling 7/30-day retention	Per-user rollup	1,360 ms	80 ms	17×
Q3 Daily active users	Daily rollup table	2,129 ms	0.5 ms	~4,000×
Q4 Revenue by cohort	Materialized view	41 ms	1.05 ms	39×
Q5 JSONB campaign attribution	GIN index	1,781 ms	416 ms	4×

Median of 5 runs for the largest tenant (about 25% of events). Results verified identical to the original queries for Q2, Q3 and Q4.

EVIDENCE · THE RETENTION QUERY

Rolling 7 and 30-day retention, sub-second

1.36 s → 80 ms

Of the users who signed up each week, how many were still active 7 and 30 days later? A per-user rollup (first and last event) replaces a scan of 10.3 million raw events, with identical results.

BRIN index

Time-range scan: 175 ms down to 48 ms

Row-Level Security

No measurable overhead on the same query

SAFETY

Security and recovery: 31 of 31 checks pass

✓ Least privilege

No app superuser. Ingest can only insert; dashboards are read-only.

✓ Row-Level Security

On all 8 tenant tables. With no tenant set, queries return nothing.

✓ Sensitive data hashed

Only an HMAC-SHA256 of the email is stored, never the address.

✓ Audit log

Triggers on tenants, plans, subscriptions and payments. Append-only.

✓ Parameterized queries

An injection probe is treated as plain data.

✓ Backup and restore

386 MB dump restored in 278 s. All 9 measures match live data.

LESSONS

Three bugs that testing caught

1 **COPY is refused under Row-Level Security**

The ingest consumer failed on every batch. Fix: a parameterized multi-row INSERT.

2 **Missing statistics on small tables**

The planner chose a bad join: the funnel query took 197 ms instead of 21 ms. Fix: ANALYZE after seeding.

3 **A misleading experiment**

RLS looked 15 times faster, but the runs used different plans. With statistics fixed, the test was fair: no measurable cost.

HONEST LIMITS

What is not done yet, and what comes next

Not built or not yet run

- Automatic failover: the replica can be promoted by hand only
- Partition maintenance is not scheduled yet
- Replica, Prometheus and Grafana are configured but not yet run here
- Q5 is still 416 ms

Next steps

- Patroni for automatic failover (recovery under 5 minutes)
- pg_cron to run partition maintenance
- Replicated Redis, or Kafka, to remove the one-second loss window
- Try TimescaleDB compression